Neuro-Fuzzy Computing - Problem-set 2nd: Teamwork

GEORGIOS MOIROTSOS - 03401
GEORGIOS MASTELLOS - 03402

December 13 - January 12, 2025

- Problem 01

F(w) =

$$w_1^2 + 10^6 w_2$$

For simplier calculations i will suppose

$$a = 10^6$$

Firstly, i find the :

$$\frac{\partial f}{\partial w_1}, \frac{\partial f}{\partial w_2}$$

$$\frac{\partial f}{\partial w_1} = 2w_1$$

$$\frac{\partial f}{\partial w_2} = 2aw_2$$

Then i will calculate the Hessian :

$$H = \begin{bmatrix} 2 & 0 \\ 0 & 2a \end{bmatrix}$$

The generall equations are :

$$w_{i+1} = w_i + \lambda_i s_i$$

where,

$$\lambda_0 = \frac{-\nabla f(w_0)^T s_0}{s_0^T H s_0}.$$

and for k = k + 1

$$s_k = -\nabla f(w_k) + \omega_k s_{k-1},$$

where:

$$\omega_k = \frac{\nabla f(w_k)^T \nabla f(w_k)}{\nabla f(w_{k-1})^T \nabla f(w_{k-1})}.$$

Calculate w1

$$s_0 = -\nabla f(w_0) = -\begin{bmatrix}6 & 6a\end{bmatrix} = \begin{bmatrix}-6 & -6a\end{bmatrix}$$

$$\lambda_0 = \frac{-\begin{bmatrix}6 & 6a\end{bmatrix}\begin{bmatrix}-6\\-6a\end{bmatrix}}{\begin{bmatrix}-6\\-6a\end{bmatrix}\begin{bmatrix}2 & 0\\0 & 2a\end{bmatrix}\begin{bmatrix}-6\\-6a\end{bmatrix}} = ... = \frac{36(1+a^2)}{72(1+a^3)} = \frac{1}{2a}$$

$$w_1 = \begin{bmatrix}3\\3\end{bmatrix} + \frac{1}{2a}\begin{bmatrix}-6\\-6a\end{bmatrix} = ... = \begin{bmatrix}2.999997\\0\end{bmatrix}$$

0.1 Calculate w2

For my ease in the calculations i supposed that:

$$w_1 = \begin{bmatrix}3\\0\end{bmatrix}$$

$$s_1 = -\nabla f(w_1) + \omega_1 s_0 = -\begin{bmatrix}6\\0\end{bmatrix} + \frac{\begin{bmatrix}6 & 0\end{bmatrix}\begin{bmatrix}6\\0\end{bmatrix}}{\begin{bmatrix}6 & 6a\end{bmatrix}\begin{bmatrix}6\\6a\end{bmatrix}}\begin{bmatrix}-6 & -6a\end{bmatrix} = ... = \begin{bmatrix}-6-6a^{-2}\\-6a^{-1}\end{bmatrix}$$

$$\lambda_1 = \frac{\begin{bmatrix}6 & 0\end{bmatrix}\begin{bmatrix}-6-6a^{-2}\\-6a^{-1}\end{bmatrix}}{\begin{bmatrix}-6-6a^{-2} & -6a^{-1}\end{bmatrix}\begin{bmatrix}2 & 0\\0 & 2a\end{bmatrix}\begin{bmatrix}-6-6a^{-2}\\-6a^{-1}\end{bmatrix}} = ... = \frac{1}{2}a^{-4}$$

$$w_2 = \begin{bmatrix}3\\0\end{bmatrix} + \frac{1}{2}a^{-4}\begin{bmatrix}-6-6a^{-2}\\-6a^{-1}\end{bmatrix} = ... = \begin{bmatrix}2.99697\\-0.00003\end{bmatrix}$$

- Problem 02

Problem's solution exists in the file **"problem02-set02.ipynb"**

Newton method for $f(w) = w_1^2 + w_2^2 * 10^6$
$\lambda_k = 1$
Methodology:

1. guess (x_0, y_0)
2. $\nabla f(x,y) = \begin{bmatrix}\frac{\partial f}{\partial x}\\\frac{\partial f}{\partial y}\end{bmatrix}$

gradient $->$ $\begin{bmatrix}2 * w_1\\2 * w_2 * 10^6\end{bmatrix}$

3. **Hessian matrix:** $H(x,y) = \begin{bmatrix} \frac{\partial^2 f}{\partial^2 x} & \frac{\partial^2 f}{\partial x * \partial y} \\ \frac{\partial^2 f}{\partial y * \partial x} & \frac{\partial^2 f}{\partial^2 y} \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 2 * 10^6 \end{bmatrix}$

Then, we continue with the Newton function:

-Loop for i <maxiter

- $g_k = \nabla f(x_k, y_k)$
- $H(x_k, y_k)$
- $x_{k+1} = x_k - H_k^{-1} * g_k$
- $y_{k+1} = y_k - H_k^{-1} * g_k$
- stop if $norm(g_k) < \epsilon$

We initilialize parameters as :

- maxiter=100
- $w_0 = [1, 1]$
- tol=$10^{-6} = \epsilon$

After running the code, we find the minimum of this function.

- Problem 03

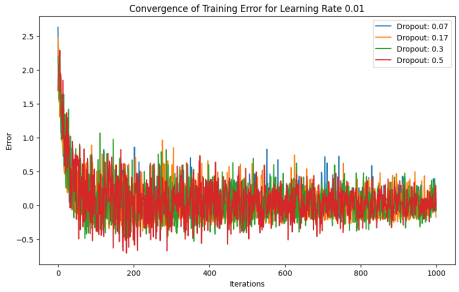


Figure 1: Enter Caption

Problem-03
In the setting with S^1= 10 and learning rate α=0.05 of **Problem-10** in the previous Problem-Set, apply the dropout technique (https://jmlr.org/papers/v15/srivastava14a.html) with dropout probability θ of hidden-layer neurons equal to θ=0.07, then with θ=0.17, and then with θ=0.3. Apply the dropout during the training phase only, and only on the hidden layer units (not of course to the input neuron). Discuss the convergence properties of the algorithm, as well as its accuracy, and contrast your findings with those of the previous exercise. Perform any additional experiments to figure out the operation of dropout as a generalization technique.

Figure 2: Enter Caption

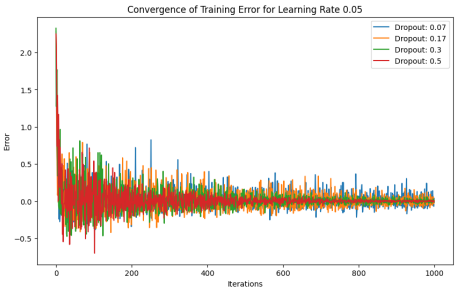


Figure 3: Enter Caption

1. **Dropout Comparison:**
   - **Low dropout (0.07, 0.17):**
     - The model learns quickly and steadily.
     - The error converges to a low value, as dropout slightly limits overfitting.
   - **High dropout (0.3, 0.5):**
     - The model takes longer to learn.
     - The error shows more fluctuations, as a large percentage of units are randomly deactivated.

Conclusions:

1. **Low Dropout:**
   - The model learns faster and reaches a stable point more easily.
   - Suitable for datasets with a low risk of overfitting.
2. **High Dropout:**
   - Increases the generalization of the model but slows down the training process.
   - Suitable for complex datasets with a high risk of overfitting.

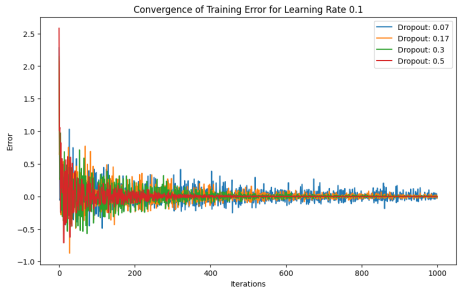


Figure 4: Enter Caption

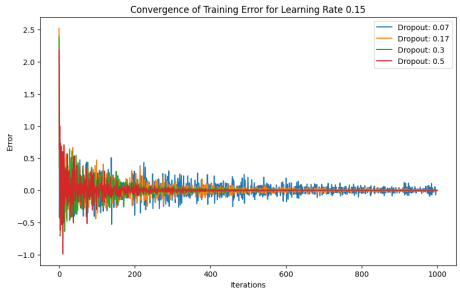


Figure 5: Enter Caption

Having the above results of the code we run into these conclusions :
Firstly we observe that as the learning rate is getting bigger the algorithm converges more roughly for different values of θ.

Problem-04

Along the lines of **Problem-10** of the previous Problem-Set, implement a MLP with S^1= 20. After training the neural network to convergence, prune a percentage (1% till 9%, step 2%) of the smallest – in magnitude – weights. Test the response (the accuracy) of the pruned MLP with respect to the unpruned version of it. Discuss the obtained results. Perform any additional experiments to figure out the operation of weight pruning as a generalization technique. Repeat the previous task, but prune now the same percentages of the largest – in magnitude – weights. Discuss again the obtained results.

Figure 6: Enter Caption

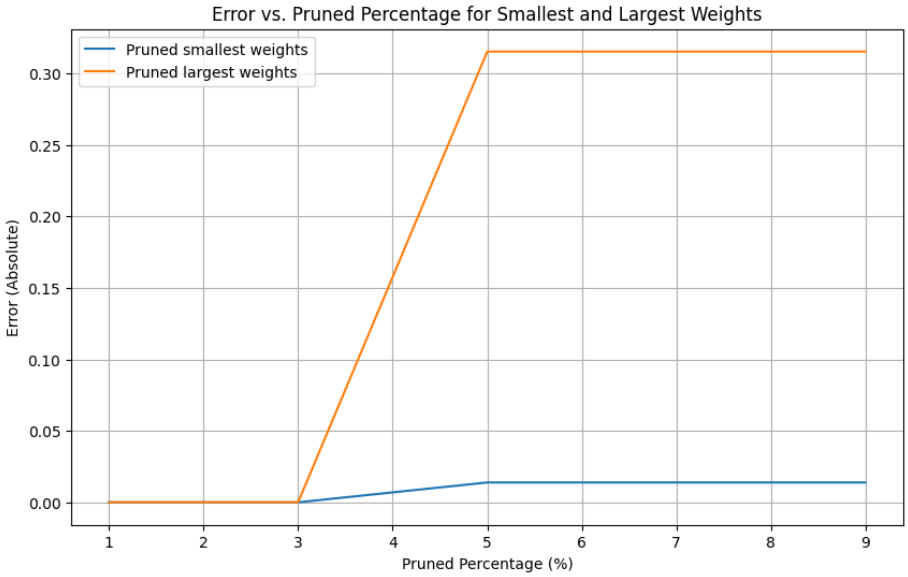


Figure 7: Results for (1%-9%)

- Problem 04

Pruning Smaller Weights (Blue Line): The error remains low and almost stable, even when up to 9% of the smaller weights are pruned.
This indicates that the smaller weights have a limited impact on the performance of the network, as they do not significantly contribute to the overall functionality.

Pruning Larger Weights (Orange Line): The error remains low for small pruning percentages (1%-3%). When more than 3% of the larger weights are pruned, the error increases sharply and stabilizes at a high value.
This suggests that the larger weights are more critical for the proper functioning of the network, and their removal leads to a significant reduction in performance.

- Problem 05

Problem's solution exists in the file **"problem05-06-set02.ipynb"**

1. Find the minimum of $f(w) = w_1^2 + w_2^2 * 10^6$ using AdaGrad for learning rate a=0.2
   Initialize w_1, w_2 randomly between $[-40, -30]$

   Steps: (loop for $i < maxiter$)

   - $g = \nabla f(x, y)$
   - $G = G + g^2$
   - $w(t) = w(t-1) - \frac{a}{\sqrt{G+\epsilon}} * g$
   - stop if $i > maxiter$ or if norm(g)<tolerance

   Initialize data:

   - maxiter=1000
   - a=0.2 (learning rate)
   - tol=10^{-6}
   - $\epsilon = 10^{-8}$

   As long as w1,w2 have been selected randomly, the results will be slightly different every time we run the code.

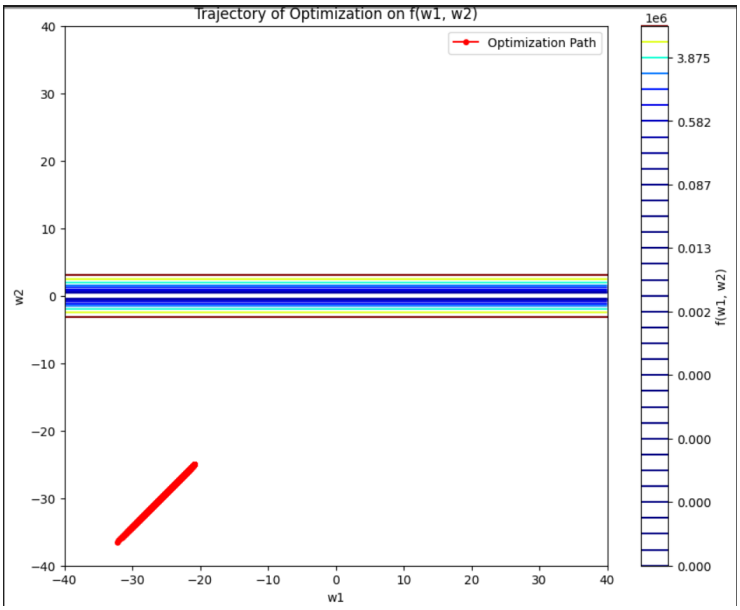


Figure 8: trajectory for a=0.2

2. We change the learning rate to a=0.02 (same code)

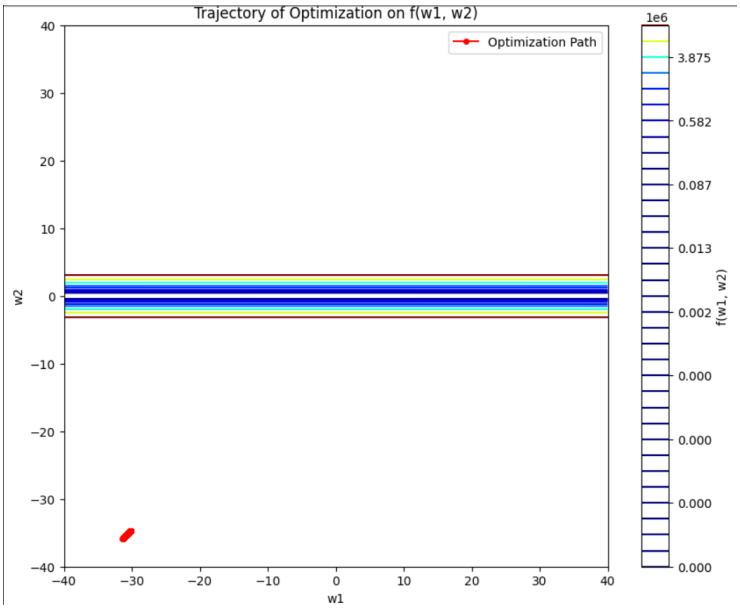


Figure 9: trajectory for a=0.02

3. We change the function to $f(w) = (w_1 - w_2)^2 + 10^6 * (w_1 + w_2)^2$ and we run the same code. We use the same values of the starting w_1, w_2 of the question 1.

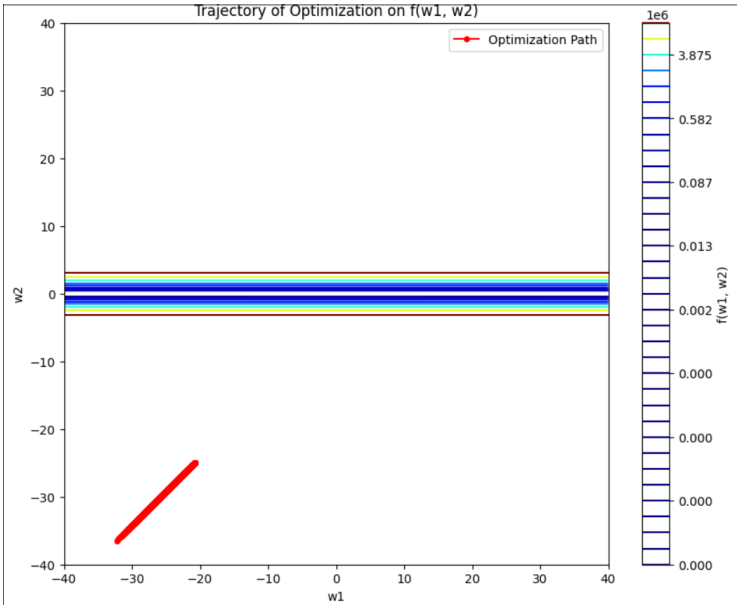


Figure 10: trajectory of new function

As we observe, trajectory plots for question 1 and 3 are the same. So, rotating the function by 45 degrees does not change the system's behavior.

- Problem 06

Problem's solution exists in the file **"problem05-06-set02.ipynb"**.
Instead of a=0.2, we use a time-varying learning rate $a(t) = \frac{a(0)}{(1+t*\frac{a(0)}{n_{epochs}})}$.
In order to compare the results between Problem 6 and Problem 5, we use the same starting values for w_1 and w_2.

- **a=0.2**

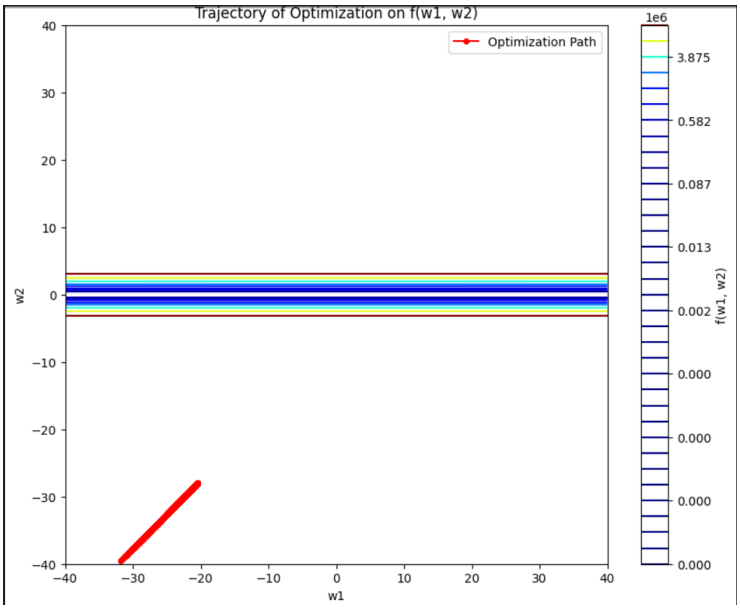


Figure 11: trajectory for a=0.2

- $a(t) = \frac{a(0)}{(1+t*\frac{a(0)}{n_{epochs}})}$

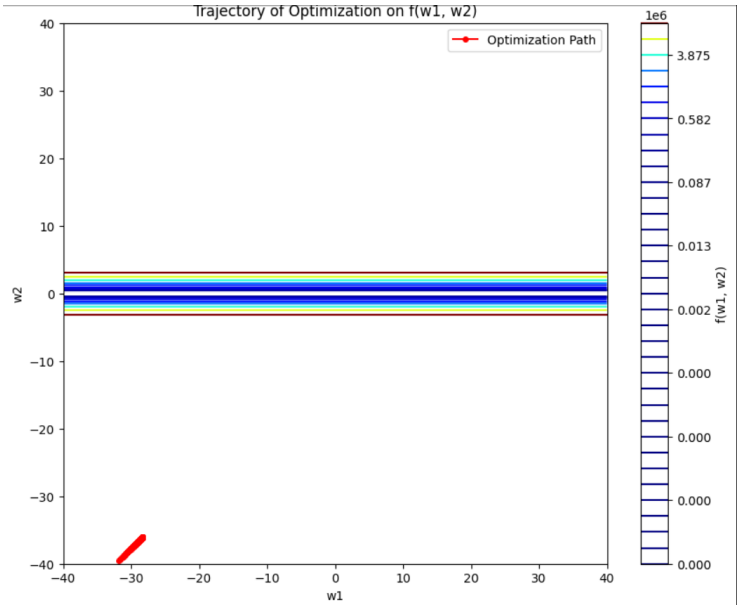


Figure 12: trajectory for $a(t) = \frac{a(0)}{(1+t*\frac{a(0)}{n_{epochs}})}$

Conclusions (observing the red line):

- red line helps visualize how the weights progress toward the minimum
- begins at the initial values of w_1 and w_2
- smaller steps between points indicate that learning rate is reducing (just like problem 6)
- for time-varying learning rate of problem 6, the convergence path is smoother and better balanced between w_1 and w_2
- For a=0.2, steps in w_2 are smaller (between [-40,-28]) than steps in w_1. This is caused because w_2 is scaled by 10^6
- The time-varying learning rate demonstrates better adaptability and efficiency in finding the minimum of $f(w_1, w_2)$

- Problem 07



Problem-07

Consider the following input image:

5	5	2	5	5
5	5	2	5	5
7	7	5	7	7
5	5	2	5	5
5	5	2	5	5

Let's assume we have the following 3x3 filters:

1	0	-1
2	0	-2
1	0	-1

1	2	1
0	0	0
-1	-2	-1

0	1	0
1	-4	1
0	1	0

Figure 13: data

A.

Stride:S=1 and **padding**: 'valid' padding

- **'valid'** padding: applying filters to an input image without adding any extra pixels around the edges
- Result: the convolution operation only processes the area of the image where the filter can fully overlap with the input data
- Since valid padding does not add any extra rows or columns, the output feature map will be smaller than the input image
- **Output Height** $= \frac{inputheight - filterheight}{Stride} + 1 = \frac{5-3}{1} + 1 = 3$
- **Output Width** $= \frac{inputwidth - filterwidth}{Stride} + 1 = \frac{5-3}{1} + 1 = 3$

In order to find the outputs, we use a program in python (**problem07-set02.ipynb**)

- **output 1:**

$$\begin{bmatrix} 11 & 0 & -11 \\ 10 & 0 & -10 \\ 11 & 0 & -11 \end{bmatrix}$$

Just for a better explanation, we include some of the computational operations of output 1:

$o11 = 5*1+5*0+2*(-1)+5*2+5*0+2*(-2)+7*1+0+5*(-1) = 11$
$o12 = 5+0-5+10+0-10+7+0-7 = 0$

$o13 = 2 + 0 - 5 + 4 + 0 - 10 + 5 + 0 - 7 = -11$
$o21 = 5 + 0 - 2 + 14 + 0 - 10 + 5 + 0 - 2 = 10$
etc

- **output 2:**

$$\begin{bmatrix} -9 & -10 & -9 \\ 0 & 0 & 0 \\ 9 & 10 & 9 \end{bmatrix}$$

- **output 3:**

$$\begin{bmatrix} -1 & 9 & -1 \\ -6 & -2 & -6 \\ -1 & 0 & -1 \end{bmatrix}$$

B. Looking at the structure of the filters, we can easily understand that:

- output 1: detect vertical edges
- output 2: detect horizontal edges
- output 3: detect diagonal edges

- Problem 08

Problem-08

Let X be an input image and K be a filter. For all (x, y) ϵ Z^2 we define T_{xy} to be the translation operator that moves every point X_{ij} in the image by x in the x-direction and by y in the y-direction, i.e.: $T_{x,y}X_{i,j} = X_{i-x,j-y}$.
Show that the convolution operator is translation-equivariant, i.e., commutes with translations: $T_{xy}X * K = T_{xy}(X * K)$.

Figure 14: Problem 8

Step 1

From above:

$$T_{xy}X_{ij} = X_{i-x,j-y}(1)$$

Convolution Origin :

$$X * K = \sum_{m=1}^{a} \sum_{n=1}^{b} X_{i+m-1,j+n-1} K_{m,n}$$

Step 2: Calculate first part of equation

$$(T_{xy}X_{ij}) * K = X_{i-x,j-y} * K = \sum_{m=1}^{a}\sum_{n=1}^{b} X_{i-x+m-1,j-y+n-1}K_{m,n}$$

Step 3: Calculate second part of equation

$$T_{xy}(X_{ij} * K) = (X * K)_{i-x,j-y} = \sum_{m=1}^{a}\sum_{n=1}^{b} X_{i-x+m-1,j-y+n-1}K_{m,n}$$

- Problem 09

Problem-09
Max-pooling can be accomplished using ReLU operations.
A. Express max(a, b) by using only ReLU operations.
B. Use this to implement max-pooling by means of convolutions and ReLU layers.
C. How many channels and layers do you need for a 2x2 convolution?

Figure 15: Enter Caption

A.

$$\max(a, b) = b + \mathbf{ReLU}(a - b)$$

If

$$a > b$$

then :

$$\max(a, b) = b + (a - b) = a$$

and if

$$a < b$$

then ReLU equals to zero, so

$$\max(a, b) = b + 0 = b$$

B.

Suppose we have a max-pooling 2x2 (a,b,c,d) then we would have these equations :

$$\max(a, b) = b + \mathbf{ReLU}(a - b)$$
$$\max(c, d) = d + \mathbf{ReLU}(c - d)$$

Thus,

$\max(a,b,c,d) = \max(\max(a,b), \max(c,d)) = \max(c,d) + \mathbf{ReLU}(\max(a,b) - \max(c,d))$

To achieve these we would use convolution layes to calculate the substractions and ReLU layers for the rest ones

C.

- Problem 10

- Input $100 * 100$, bias=1
- 1st layer: Convolution layer with 50 $5 * 5$ filters
- 2nd layer: Convolution layer with 300 $5 * 5$ filters
- 3rd layer: max pooling $->$ down samples layer 2 $(50 * 50)$
- 4th layer: Dense layer with 200 units
- 5th layer: Dense layer with 300 units
- 6th layer: Single output unit

Solution:

1. **1st Convolution Layer**
   $5 * 5 + 1 = 26$ weights per neuron
   $50 * 26 = 1300$ total weights for layer 1 **(1)**
2. **2nd Convolution Layer**
   $5 * 5 * 50 + 1 = 1251$ weights per neuron
   $1251 * 300 = 375,300$ total weights for layer 2 **(2)**
3. **3rd layer - Max Pooling**
   factor $4 -> 2 * 2$ window size
   - Max pooling does not affect weights of a neural network, because it does not contain any trainable parameters
   - total weights for third layer is 0 **(3)**
4. **4th layer - Dense Layer**
   - fully connected layer
   - each neuron in the 4th layer is connected to every neuron in the previous layer
   - J=K=50, M=0, S=1, $\frac{J-M}{S} = 50$

- $50 * 50 * 300 = 750,000$ weights per neuron
- $750,000 * 200 + 200(biases) = 150,000,000$ total weights for 4th layer **(4)**

5. **5th layer - Dense Layer**
   - number of units stands for the number of neurons in that layer
   - $200 * 300 + 300(biases) = 60,000 + 300 = 60,300$ total weights for 5th layer **(5)**
6. **6th layer - Single Output**
   - 300+1=301 neurons in 5th layer
   - $301 * 1 = 301$ total weights for 6th layer **(6)**

From (1),(2),(3),(4),(5),(6) **total weights of the network:**
$1300 + 375,300 + 0 + 150,000,200 + 60,300 + 301 =$ **150,437,401** weights

- Problem 11

Making fast convolutions with two possible ways:

1. reading k-wide strip and computing the 1-wide output strip and value at a time
2. read k+Δ wide strip and compute a $\Delta-$wide output strip

Let's say we have an input strip of length 10 and a k=5.

1. We'll have a kernel of size 5.
   We will apply the kernel to each segment of the input strip, moving one position at a time, and compute the output value at each position.
   For example, input strip = [1,2,3,4,5,6,7,8,9,10] and kernel = [0.1,0.2,0.3,0.4,0.5].
   - **Output in first position:**
     (1×0.1)+(2×0.2)+(3×0.3)+(4×0.4)+(5×0.5) = 5.5
   - **Output in second position:**
     (2×0.1)+(3×0.2)+(4×0.3)+(5×0.4)+(6×0.5)=7
   - **Output in third position:**
     (3×0.1)+(4×0.2)+(5×0.3)+(6×0.4)+(7×0.5)=8.5
   - **Output in fourth position:**
     (4×0.1)+(5×0.2)+(6×0.3)+(7×0.4)+(8×0.5)=10
   - **Output in fifth position:**
     (5×0.1)+(6×0.2)+(7×0.3)+(8×0.4)+(9×0.5)=11.5
   - **Output in sixth position:**
     (6×0.1)+(7×0.2)+(8×0.3)+(9×0.4)+(10×0.5)=13

- **Final output:** After applying the kernel to all possible segments of the input strip, the resulting output strip is: [5.5,7,8.5,10,11.5,13]

2. Again, we have an input strip of length 10 and k=10.
   Now, we apply a Δ, let's say $\Delta = 2$. The kernel size will be $k+\Delta = 5+2 = 7$. We will place the kernel over the first segment of the input strip and do the same calculations as before. Once we find the first output value, we **move the kernel Δ (2) positions to the right** and repeat the same procedure. For example, lets say kernel = [0.1,0.2,0.3,0.4,0.5,0.6,0.7]

   - **First output value:**
     (1×0.1)+(2×0.2)+(3×0.3)+(4×0.4)+(5×0.5)+(6x0.6)+(7x0.7) = ...
   - **Second output value:**
     We move 2 positions to the right
     (3×0.1)+(4×0.2)+(5×0.3)+(6×0.4)+(7×0.5)+(8x0.6)+(9x0.9)=...
   - Continue this process until the kernel can no longer move $\Delta = 2$ positions to the right

Choosing Δ:

- $\Delta's$ size affects significantly output's result. A larger Δ means that kernel moves up to the right faster and this leads to less calculations and a different result than a smaller Δ which leads to more calculations.
- A larger Δ means less calculations and, as a result, less computational cost

- Problem 12

From the plot that is given to us, we considered that the two circles have centers:

$$(-1.5, 0.5)(1, 1)$$

So

$$W_1 = \begin{bmatrix} -1.5 \\ 0.5 \end{bmatrix}$$

and

$$W_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\sigma = \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix}$$

as these are the radius of the circles. Thus,

$$a_i^1 = \text{radbas}(\|\mathbf{w}_i^1 - \mathbf{p}\| b_i^1)$$



Problem-12
Design an RBF network to perform the classification illustrated in the following figure. The network should produce a positive output whenever the input vector is in the shaded region and a negative output otherwise.

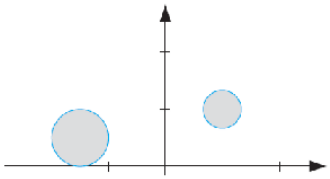


Figure 16: Enter Caption

Where we use the gaussian function and with the appropriate p we are able to find a1.
Because we couldn't find a solution in the paper we followed the idea of the example we solved in the lecture. We took many vectors in the field p1,p2,..p8 and we use them to come up to the solution.
For this exercise also used python code, in order to plot the two 'mountains' and be able to find the bias for a2, the code is also in the folder in order to help in the understanding of the exercise. It should also be mentioned that for the 3d plot chatgpt helps was neccessary.

Taken everything into account from the above we used

$$W_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$b_2 = -0.55$$

to make the ideal RBF .

- Problem 13

Problem's solution exists in the file **"problem13-set02.ipynb"**.

Implement the gradient descent algorithm for a RBF network. Train the network to approximate the function $g(p) = 1 + sin(\frac{p*\pi}{3})$ for $-3 <= p <= 3$.

RBF is a specialized type of NNs characterized by their unique architecture and ability to approximate complex functions effectively.

Architecture:

- **Input Layer:** receives input data, with the number of neurons corresponding to the dimensionality of the input
- **RBF Layer (hidden layer):** The hidden layer is composed of k radial basis functions, often Gaussians functions.
- **Output layer:** generates the final output. The output is calculated as a weighted sum of the inputs from the radial basis function layer

Algorithm Steps:

1. **Initialization:**
   - select 20 p data points randomly, with respect to the function $g(p) = 1 + sin(\frac{p*\pi}{3})$, for $-3 <= p <= 3$
   - I have **as many neurons as the k centers**
   - initialize randomly weights and biases (dimensionality [kcenters, 1]). Weights in the [-0.5,0.5] and the biases in the [0,0.1]
   - learning rate a=0.01
   - initialize centers "c" in the space [-3,3] randomly ((dimensionality [kcenters, 1])
   - initialize spreads σ with repsect to $\sigma = \frac{randompointbetween[-3,3]}{\sqrt{2*kcenters}}$
2. **Compute Hidden Layer Activation**:
   - compute hidden layer units with respect to $h_{i(p)} = exp(-\frac{(p-c_i)^2}{2*c_i^2})$ (dimensionality [kcenter,20])
   - compute network's output with respect to $\hat{g}(p) = \sum_{i=1}^{k} w_i h_i(p) + b$ (as many output values as the data points)
   - compute the loss function with respect to the **Mean Squared Error**, Loss (MSE) $= \frac{1}{N} \sum_{j=1}^{N} \left(g(p_j) - \hat{g}(p_j)\right)^2$
   - smaller loss means the network's predictions are closer to the true values. As long as the initialization takes place randomly, we run the code until the loss value <= 0.5.
3. **Update gradients - training:**

- **gradient weights:** $\frac{\partial \text{Loss}}{\partial w_i} = -\frac{2}{N}\sum_{j=1}^{N}(g(p_j) - \hat{g}(p_j)) \cdot h_i(p_j)$
- **gradient centers** (c_i): $\frac{\partial \text{Loss}}{\partial c_i} = -\frac{2}{N}\sum_{j=1}^{N}(g(p_j) - \hat{g}(p_j)) \cdot w_i \cdot h_i(p_j) \cdot \frac{p_j - c_i}{\sigma_i^2}$
- **gradient bias:** $\frac{\partial \text{Loss}}{\partial b} = -\frac{2}{N}\sum_{j=1}^{N}(g(p_j) - \hat{g}(p_j))$. Bias will be a single value.
- **gradient spreads:** $\frac{\partial \text{Loss}}{\partial \sigma_i} = -\frac{2}{N}\sum_{j=1}^{N}(g(p_j) - \hat{g}(p_j)) \cdot w_i \cdot h_i(p_j) \cdot \frac{(p_j - c_i)^2}{\sigma_i^3}$

4. **Update parameters for layer 2:**
   Gradient Descent rule: $\theta = \theta - a * \frac{\partial loss}{\partial \theta}$.
   For example, $w_i = w_i - a * \frac{\partial loss}{\partial w_i}$. Same for centers, spreads and bias.
5. Find final output points using the type $a_2 = w_2 * a_1 + b_2$ and using the updated weights and bias.
6. **Plot network response and training points on the same plot**
7. **Run the code for kcenter 4,8,10**

- Problem 14

I have to calculate the

$$n_i = Wp$$

and to find the max value of n

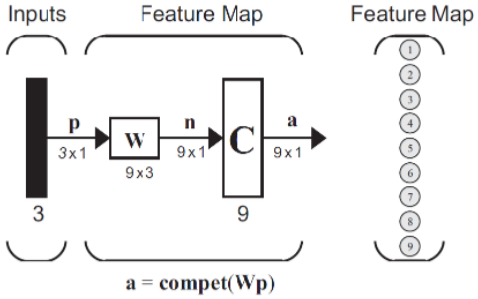
$$n = W_p = \begin{bmatrix} 0.41 \cdot 0.67 + 0.41 \cdot 0.07 + 0.82 \cdot 0.74 \\ 0.45 \cdot 0.67 + 0.00 \cdot 0.07 + 0.89 \cdot 0.74 \\ 0.41 \cdot 0.67 - 0.41 \cdot 0.07 + 0.82 \cdot 0.74 \\ 0.00 \cdot 0.67 + 0.45 \cdot 0.07 + 0.89 \cdot 0.74 \\ 0.67 \cdot 0.00 + 0.00 \cdot 0.07 + 1.00 \cdot 0.74 \\ 0.00 \cdot 0.67 - 0.45 \cdot 0.07 + 0.89 \cdot 0.74 \\ -0.41 \cdot 0.67 + 0.41 \cdot 0.07 + 0.82 \cdot 0.74 \\ -0.45 \cdot 0.67 + 0.00 \cdot 0.07 + 0.89 \cdot 0.74 \\ 0.41 \cdot 0.67 - 0.41 \cdot 0.07 + 0.82 \cdot 0.74 \end{bmatrix}$$

$$n = \begin{bmatrix} 0.9102 \\ 0.9601 \\ 0.8528 \\ 0.6901 \\ 0.74 \\ 0.6271 \\ 0.3608 \\ 0.3571 \\ 0.3034 \end{bmatrix}$$



Problem-14

Consider the following competitive neural network and the respective one-dimensional feature map.



If the initial weight matrix is the following:

$$\mathbf{W} = \begin{bmatrix} 0.41 & 0.45 & 0.41 & 0 & 0 & 0 & -0.41 & -0.45 & -0.41 \\ 0.41 & 0 & -0.41 & 0.45 & 0 & -0.45 & 0.41 & 0 & -0.41 \\ 0.82 & 0.89 & 0.82 & 0.89 & 1 & 0.89 & 0.82 & 0.89 & 0.82 \end{bmatrix}^T$$

Train the network on the vector below for one iteration with learning rate α=0.1, and a neighborhood of radius equal to 1.

$$\mathbf{p} = \begin{bmatrix} 0.67 \\ 0.07 \\ 0.74 \end{bmatrix}.$$

Figure 17: Enter Caption

Because

$$n_2$$

is the maximum value we will change the weights of 1,2,3.
Calculate the new W for

$$W_{1,2,3}$$

New:

$$W_2 = W_{2old} + a(p_2 - W_{2old})$$

$$W_2 = \begin{bmatrix} 0.45 \\ 0 \\ 0.89 \end{bmatrix} + 0.1(\begin{bmatrix} 0.67 \\ 0.07 \\ 0.74 \end{bmatrix} - \begin{bmatrix} 0.45 \\ 0 \\ 0.89 \end{bmatrix})$$

$$W_2 = ... = \begin{bmatrix} 0.472 \\ 0.007 \\ 0.875 \end{bmatrix}$$

Likewise for

$$W_{1,3}$$

$$W_1 = \begin{bmatrix} 0.436 \\ 0.376 \\ 0.362 \end{bmatrix}$$

$$W_3 = \begin{bmatrix} 0.436 \\ 0.458 \\ 0.362 \end{bmatrix}$$